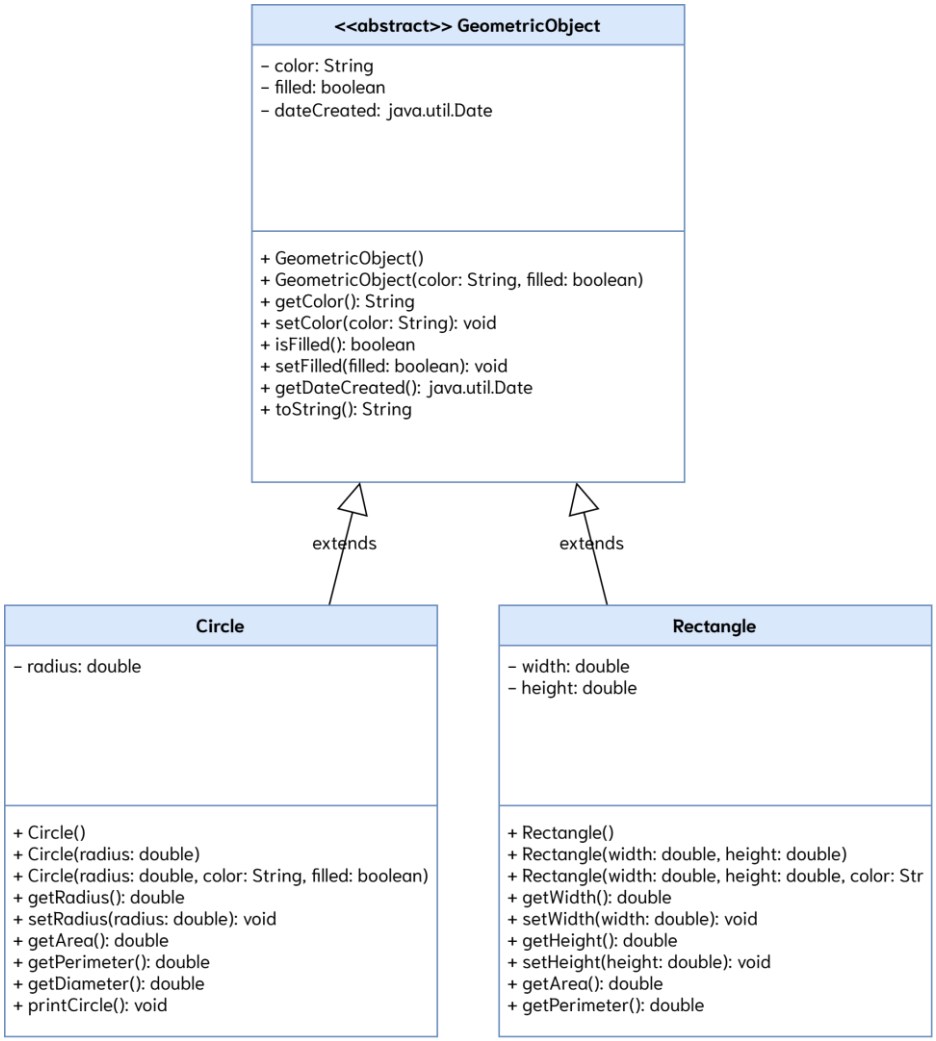
แบบฝึกหัดปฏิบัติการคาบที่ 6: Inheritance

1. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสืบทอดคลาสจากตัวอย่างต่อไปนี้




```java
public abstract class GeometricObject {
    private String color = "white";
    private boolean filled;
    /** Default constructor */
    protected GeometricObject() {
    }
    /** Convenience constructor */
    protected GeometricObject(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }
    public String getColor() {
        return color;
    }
    public void setColor(String color) {
        this.color = color;
    }
    public boolean isFilled() {
        return filled;
    }
    public void setFilled(boolean filled) {
        this.filled = filled;
    }
    public abstract double findArea();
    public abstract double findPerimeter();
}
```

```java
public class Circle extends GeometricObject {
    private double radius;

    /** Default constructor */
    public Circle() {
        this(1.0);
    }

    /** Radius convenience constructor */
    public Circle(double radius) {
        this(radius, "white", false);
    }

    /** Convenience constructor for all properties */
    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    /**
     * Return the radius
     * @return radius Current radius of Circle
     */
    public double getRadius() {
        return radius;
    }

    /**
     * Set the radius of the circle
     */
    public void setRadius(double radius) {
        this.radius = radius;
    }

    /**
     * Returns the area of the current circle
     * Implemention of abstract method in GeometricObject
     * @return area of the circle
     */
    public double findArea() {
        return radius * radius * Math.PI;
    }

    /**
     * Returns the perimeter of the current circle
     * Implemention of abstract method in GeometricObject
     * @return perimeter of the circle
     */
    public double findPerimeter() {
        return 2 * radius * Math.PI;
    }

    /**
     * Provide a string representation of the object
     */
    public String toString() {
        return "Circle: radius = " + radius;
    }
}
```

1.1 ให้อธิบายลำดับการเรียกใช้ constructor เมื่อสร้างวัตถุจากคลาส Circle

1. Circle() ถูกเรียก และพบ this(1.0) จึงเรียก Circle(double radius) แล้ว
2. Circle(double radius) พบ this(radius, "white", false) จึงเรียก Circle(double radius, String color, boolean filled) แล้ว
3. Circle(double radius, String color, boolean filled) เรียก super(color, filled) เพื่อเรียก constructor ของคลาสแม่ GeometricObject
4. GeometricObject(String color, boolean filled) กำหนด color เป็น "white" และ filled เป็น false
5. กลับมาที่ Circle(double radius, String color, boolean filled) แล้วกำหนด radius เป็น 1.0

1.2 ให้สร้างคลาส Rectangle ตาม UML Class Diagram ด้านบน

```
1 | public class Rectangle extends GeometricObject {
2 |     private double width;
3 |     private double height;
4 |
5 |     public Rectangle() {
6 |         this(1.0, 1.0);
7 |     }
8 |
9 |     public Rectangle(double width, double height) {
10|         this(width, height, "white", false);
11|     }
12|
13|     public Rectangle(double width, double height,
14|                      String color, boolean filled) {
15|         super(color, filled);
16|         this.width = width;
17|         this.height = height;
18|     }
19|
20|     public double getWidth() {
21|         return width;
22|     }
23|
24|     public void setWidth(double width) {
25|         this.width = width;
26|     }
27|
28|     public double getHeight() {
29|         return height;
30|     }
31|
32|     public void setHeight(double height) {
33|         this.height = height;
34|     }
35|
36|     public double getArea() {
37|         return width * height;
38|     }
39|
40|     public double getPerimeter() {
41|         return 2 * (width + height);
42|     }
43|
```

```
44|     @Override
45|     public double findArea() {
46|         return getArea();
47|     }
48|
49|     @Override
50|     public double findPerimeter() {
51|         return getPerimeter();
52|     }
53| }
```

2. จงหาข้อผิดพลาดของส่วนของโปรแกรมต่อไปนี้

```
public class Circle{
   private double radius;

   public Circle(double radius){
      radius=radius;
   }
   public double getRadius(){
      return radius;
   }
   public double findArea(){
      return radius*radius*Math.PI;
   }
}
class Cylinder extends Circle{
   private double length;

   Cylinder(double radius, double length){
      Circle(radius);
      length=length;
   }
}
```

จุดที่หนึ่ง `radius` ที่เขียนทั้งคู่นั้นนับว่าเป็นตัวแปร parameter `radius` ไม่ใช่ instance variable `radius`

จุดที่สอง เราไม่สามารถเรียก constructor ของคลาสแม่แบบนั้นได้ ต้องใช้ `super(radius)` และต้องพิมพ์เป็นบรรทัดแรกของ constructor ลูก

จุดที่สาม `length` ที่เกิดปัญหาเดียวกับจุดแรก

3. จงหาผลลัพธ์การรันโปรแกรมต่อไปนี้

3.1

```
class A{
   public A(){
      System.out.println("The no-arg constructor of A is
invoked");
   }
}
class B extends A{
   public B(){

   }
}
public class C{
   public static void main(String[] args){
     B b =new B();
   }
}
```

ผลลัพธ์การรันโปรแกรม

```
The no-arg constructor
of A is invoked
```

3.2

Animal.java

```
1 | public class Animal {
2 |     public Animal() {
3 |         System.out.println("A new animal has been created!");
4 |     }
5 |
6 |     public void sleep() {
7 |         System.out.println("An animal sleeps...");
8 |     }
9 |
10|     public void eat() {
11|         System.out.println("An animal eats...");
12|     }
13| }
```

Bird.java

```
1 | public class Bird extends Animal {
2 |     public Bird() {
3 |         super();
4 |         System.out.println("A new bird has been created!");
5 |     }
6 |
7 |     @Override
8 |     public void sleep() {
9 |         System.out.println("A bird sleeps...");
10|     }
11|
12|     @Override
13|     public void eat() {
14|         System.out.println("A bird eats...");
15|     }
16| }
```

Dog.java

```java
public class Dog extends Animal {
    public Dog() {
        super();
        System.out.println("A new dog has been created!");
    }

    @Override
    public void sleep() {
        System.out.println("A dog sleeps...");
    }

    @Override
    public void eat() {
        System.out.println("A dog eats...");
    }
}
```

MainClass.java

```java
public class MainClass {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Bird bird = new Bird();
        Dog dog = new Dog();

        System.out.println();

        animal.sleep();
        animal.eat();

        bird.sleep();
        bird.eat();

        dog.sleep();
        dog.eat();
    }
}
```

ผลลัพธ์การรันโปรแกรม

```
A new animal has been created!
A new animal has been created!
A new bird has been created!
A new animal has been created!
A new dog has been created!

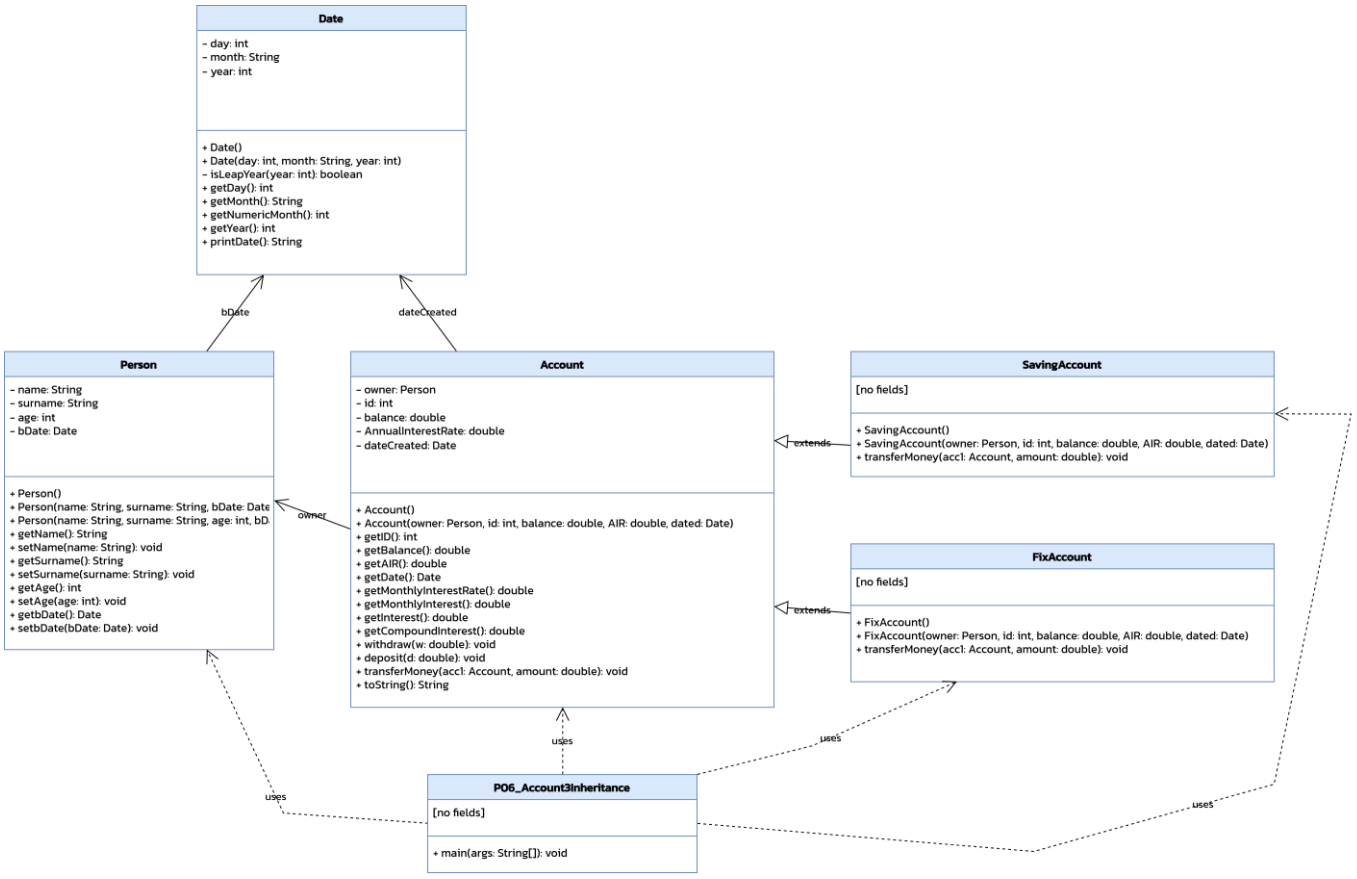
An animal sleeps...
An animal eats...
A bird sleeps...
A bird eats...
A dog sleeps...
A dog eats...
```

4. จากโปรแกรมด้านล่างต่อไปนี้ ข้อใดคือ Overriding ที่สามารถนำมาเติมลงในบรรทัดที่ 9 ของโปรแกรมได้โดยไม่ทำให้เกิดข้อผิดพลาดขึ้นกับการทำงานของโปรแกรม อธิบายเหตุผลประกอบ

```
1 | class SuperClass {
2 |     private int num = 1;
3 |
4 |     protected int getNumber() {
5 |         return num;
6 |     }
7 | }
8 |
9 | class Subclass extends SuperClass {
10|     private int num = 10;
11|
12|     // Overriding method
13|     public static void main(String[] args) {
14|         Subclass s = new Subclass();
15|         System.out.println(s.getNumber());
16|     }
17| }
```

#	โปรแกรม	สามารถนำมาเติมลงในบรรทัดที่ 12 ของโปรแกรมได้โดยไม่ทำให้เกิดข้อผิดพลาดหรือไม่	อธิบายเหตุผลประกอบ
1	`protected int getNumbers(){ return num+5; }`	ไม่เกิดข้อผิดพลาด	ชื่อเมธอดไม่เหมือนกัน นับว่าเป็นคนละเมธอด
2	`protected long getNumber(){ return num+5; }`	เกิดข้อผิดพลาด	เมธอดนี้ไม่สามารถ override เมธอดแม่ได้ เพราะว่า return type เป็น long
3	`protected int getNumber(){ return num+5; }`	ไม่เกิดข้อผิดพลาด	ชื่อเมธอดเหมือนกัน ลำดับและประเภท argument และ return type เหมือนกันหมด จึงเกิดการ override
4	`public int getNumber(){ return num+5; }`	ไม่เกิดข้อผิดพลาด	เหมือนกับข้างบน ()แต่ว่าทำได้เพราะเป็นการขยาย scope ของการเข้าถึงเมธอด
5	`protected int getNumber(int num){ return num+5; }`	ไม่เกิดข้อผิดพลาด	ลำดับและประเภท argument ต่างกัน แต่ชื่อเมธอดเหมือนกัน นี่จึงเป็นการ overload
6	`int getNumber(){ return num+5; }`	เกิดข้อผิดพลาด	การเข้าถึงแบบ default เป็น private ทำให้เกิดการลด scope ของการเข้าถึงเมธอด ซึ่งไม่สามารถทำได้ถ้าจะ override
7	`private int getNumber(){ return num+5; }`	เกิดข้อผิดพลาด	เกิดการลด scope ของการเข้าถึงเมธอด ซึ่งไม่สามารถทำได้ถ้าจะ override

6. Class Diagram



7. Class Diagram

